CS 530 Lab Notebook

Lab 3

Name: Jarren Calizo
OdinID: calizo
Date: October 19, 2025

Table of Contents

Lab Overview 2
Lab 3.1 Python Flask Guestbook 3
Step 1-5: Python Flask Setup and Running The Application 3
Step 1 - Python Flask Setup 3
Step 2 - Model Configuration 3
Step 3 - Controller Implementation 3
Step 4 - View and Templates 3
Step 5 - Running the Application 4
1. Add an entry that includes your PSU e-mail address in it and the message "python/flask guestbook". Take a screenshot of the resulting page for your lab notebook. 4
Lab 3.2ag SQL 5
Step 1: SQL, Cloud SQL, RDS 5
Step 2: SQL Quiz 6
1. Take the quiz and include a screenshot with your OdinID on it of the "Check your answers" page at the end of the quiz. You do not have to be perfect, but check the ones you got incorrect. 6
Step 3: GCP Cloud SQL 7
1. What are the names of the tables that are created? 7
2. What are the primary keys of each table? 8
3. What data (e.g. columns) does the Accommodation table hold? 8
4. Assuming the column data is ordered as in the DDL, list the attributes and their values for each accommodation in Dublin. 9
Step 4: Cloud SQL Instance Creation (Option #1) 10
Step 5: Cloud SQL Instance Creation (Option #2) 11
Step 6: Cloud SQL Importing Data 12
Step 7: Cloud SQL from Cloud Shell 13
1. Take screenshots of the output of each query for your lab notebook. 14
Step 8: Cloud SQL Cleanup 16
1. Take a screenshot showing no instances are running. 17
Step 9: AWS RDS 18
Step 10: Cloud Shell Network Address 18
Step 11: RDS Security Group Creation 18
Step 12: RDS Instance Creation 19
Step 13: Viewing the Instance 21
Step 14: RDS Test Connection 21
Step 15: RDS Cleanup 22
1. Show a screenshot of the successful connection similar to below that includes your OdinID. 22
Step 16: RDS Cleanup 22
Lab 3.3 sqlite3 Guestbook 24
Step 1: sqlite3 Guestbook Overview 24
Step 2: gbmodel Package 24
Step 3: Presenter Architecture 24
Step 4: Running the Application 25
1. Take a screenshot of the resulting page for your lab notebook. 25
Step 5: sqlite3 Database Verification 26
1. Then, within the sqlite client, perform the following commands and take a screenshot of their output to include in your lab notebook. 27

Lab Overview

Lab 3 served as an integrative exploration of how web applications interact with persistent data layers across both local and cloud environments. It unified three major dimensions of web systems development: local web frameworks (Flask), cloud-hosted databases (GCP Cloud SQL and AWS RDS), and lightweight embedded databases (sqlite3). Together, these components revealed the complete lifecycle of modern web data management - from transient in-memory structures to scalable cloud-backed architectures.

The journey began with Lab 3.1, which introduced Flask's Model-View-Controller paradigm through a basic guestbook application. Here, the focus was on modular design and clarity - Python handled the logic, HTML templates handled presentation, and Flask's routing connected the two. Each layer echoed the principle of separation of concerns, demonstrating how even a minimal application embodies the structure of full-scale web systems. Running the guestbook locally showcased live request handling, form submission, and data rendering, emphasizing the simplicity and elegance of Flask's design.

The second phase, Lab 3.2ag, transitioned from local development to cloud infrastructure, diving deep into SQL fundamentals and managed database services. Beginning with schema inspection and SQL query execution, this section bridged theory with practice - reinforcing relational design concepts such as primary keys, normalization, and entity relationships. The exercises on Google Cloud SQL involved creating storage buckets, importing datasets, and executing queries through Cloud Shell, revealing the seamless integration between storage and computation layers in cloud ecosystems. The mirrored implementation on AWS RDS highlighted the universality of SQL and the parallelism between cloud providers - both enabling secure, scalable, and accessible data persistence. Crucially, the teardown and cleanup steps instilled the discipline of resource lifecycle management, reflecting real-world DevOps responsibility in cost control and environmental hygiene.

Finally, Lab 3.3 grounded the experience by returning to a self-contained architecture using sqlite3 within a Flask MVP (Model-View-Presenter) structure. This marked the transition from transient data to persistence, as the guestbook evolved to maintain entries across server restarts. The gbmodel package encapsulated initialization and table creation logic, the presenters handled user interactions, and the sqlite database provided a durable storage layer. Together, they embodied how scalable modularity can coexist with minimalism - forming the blueprint of many production-ready applications.

Across all three sections, Lab 3 illuminated the interconnectedness of web frameworks, database systems, and cloud infrastructure. It was more than a set of exercises - it was an architectural narrative of how data flows, persists, and scales across environments. From the comfort of localhost to the elasticity of the cloud, the lab stressed a recurring theme: that the beauty of web systems lies not just in their interfaces, but in the orchestration of their layers beneath.

Lab 3.1 Python Flask Guestbook

Step 1-5: Python Flask Setup and Running The Application

Step 1: Python Flask Setup

I began by remotely connecting to my Linux VM and cloning the course repository from GitHub. Navigating into cs430-src/01_mvc_pylist revealed a concise Model-View-Controller (MVC) structure. Each component was neatly divided: Model.py and model_pylist.py for the backend logic, app.py as the controller, and templates/ with static/ for the view layer. This separation made the architecture modular and readable. The structure emphasized the Flask philosophy of simplicity - minimal boilerplate yet clear abstraction boundaries.

Step 2: Model Configuration

Before running the application, I reviewed the model implementation. The Model.py defined an abstract base class, while model_pylist.py extended it to store entries in an in-memory Python list. The insert function appended the user's name, email, message, and the current date, whereas the select function returned the entire guest list. Because this data lives only in memory, every time the Flask server restarts, the guestbook resets. This design lays the groundwork for later transitioning to persistent backends like SQLite or MySQL.

Step 3: Controller Implementation

The controller, defined in app.py, handled both rendering and form submission. When running the app, Flask created a local web server on port 5000. The index() route displayed all guestbook entries by looping over the results of model.select() and rendering them in index.html. The /sign route processed POST requests, passing form inputs to model.insert() before redirecting back to the main page. Seeing this logic in action highlighted Flask's intuitive routing system - lightweight yet capable of managing full-stack workflows.

Step 4: View and Templates

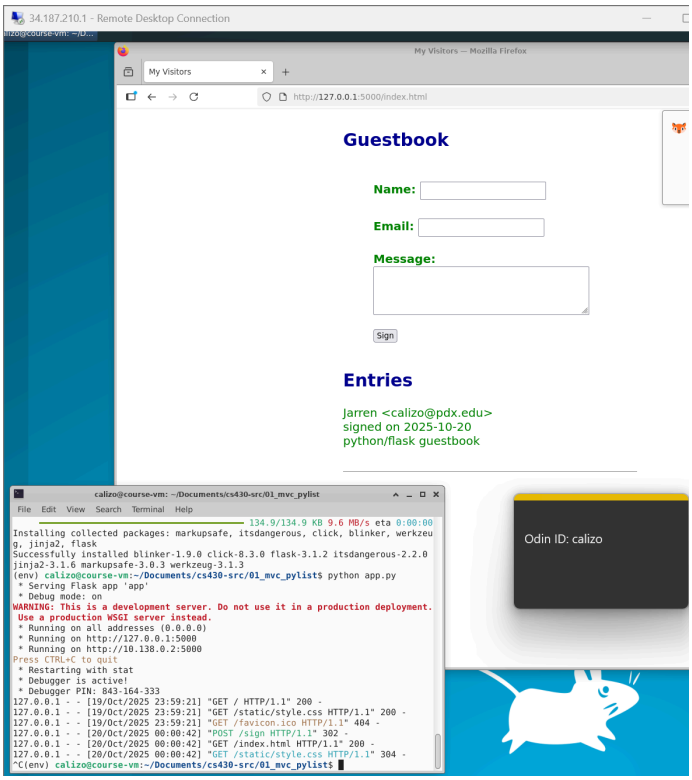
The templates embodied the visual simplicity of Flask's Jinja2 engine. The layout.html provided a base layout with linked CSS and a content block for inherited templates. The index.html extended this base, adding the form and dynamic rendering for guest entries. Each entry displayed the user's name, email, date signed, and message. The

clear structure showed how Flask efficiently integrates logic and HTML through Jinja syntax, bridging backend data with front-end representation in just a few lines of templating code.

Step 5: Running the Application

I installed the required dependencies, created a Python virtual environment, and activated it to isolate package installations. With pip install -r requirements.txt, Flask and its supporting libraries were installed. Running python app.py initialized the development server, indicated by the log messages in the terminal.

1. Add an entry that includes your PSU e-mail address in it and the message "python/flask guestbook". Take a screenshot of the resulting page for your lab notebook.



After launching the application, I visited http://127.0.0.1:5000 in Firefox and filled in my PSU email along with the message "python/flask guestbook." Upon submission, the page refreshed to display my entry in the "Entries" section, confirming that the model correctly handled data insertion. The server console concurrently logged GET and POST requests, showing the full Flask request lifecycle in real-time. Finally, I terminated the server with Ctrl+C, marking the completion of the exercise.

Insight
This lab served as a clean introduction to Flask's MVC pattern - combining backend logic, templating, and routing into one coherent web framework. Implementing a working guestbook demonstrated the power of abstraction between components while reinforcing essential web server operations. Even with a lightweight model, the app captured the essence of full-stack development - data flow, user interaction, and response rendering - all within a few files of readable Python code.

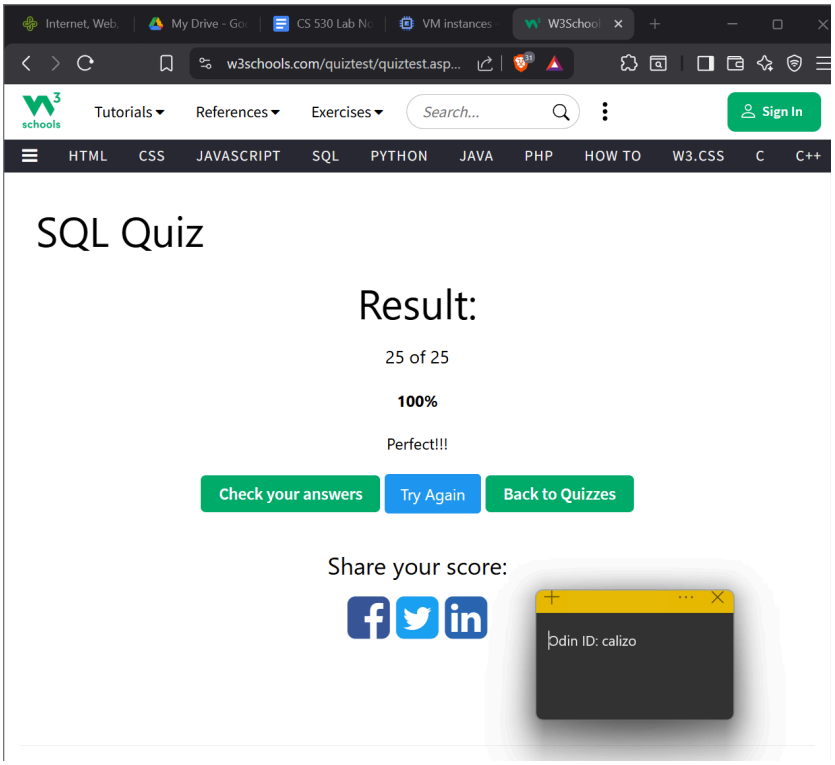
Lab 3.2ag SQL

Step 1: SQL, Cloud SQL, RDS

This step served as an introduction to the structured and transactional nature of SQL-based databases. SQL, or Structured Query Language, provides a standardized way to define, manipulate, and query data within relational tables. The key principles - atomicity, consistency, isolation, and durability - ensure the integrity and reliability of operations even across failures. In modern development environments, platforms like Google Cloud SQL and AWS RDS ease the process by hosting managed SQL databases, freeing developers from handling manual installation, scaling, and backups. This background contextualized how SQL fits into cloud systems and why it remains foundational for backend data management.

Step 2: SQL Quiz

1. Take the quiz and include a screenshot with your OdinID on it of the "Check your answers" page at the end of the quiz. You do not have to be perfect, but check the ones you got incorrect.



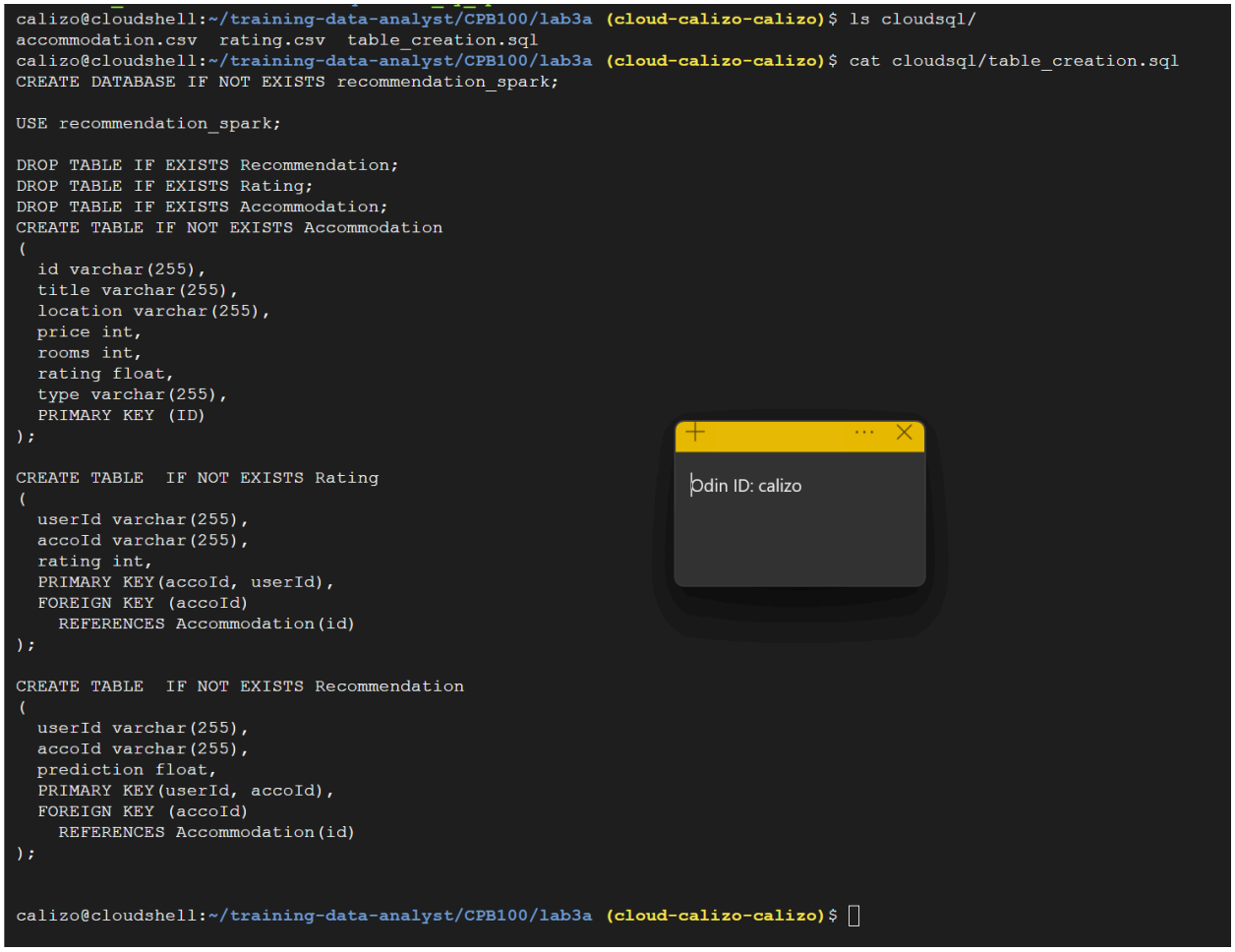
After visiting https://www.w3schools.com/sql/sql_quiz.asp, I completed the SQL quiz, confirming full comprehension of the fundamental SQL concepts tested. My OdinID was displayed beside the result page to authenticate completion.

The questions covered essential SQL operations, including data extraction with SELECT, insertion via INSERT INTO, updates with UPDATE, and deletions through DELETE. It also reinforced understanding of logical operators such as AND, OR, and pattern matching with LIKE, as well as the importance of sorting (ORDER BY), distinct selection (SELECT DISTINCT), and joins (INNER JOIN).

Completing this quiz refreshed the core syntax and mental model of how SQL statements interact with relational tables. It served as a practical checkpoint to ensure readiness for upcoming labs involving real database interaction through Python and Flask integrations.

Step 3: GCP Cloud SQL

In this step, I set up a managed MySQL environment on Google Cloud Platform using Cloud Shell and verified my dataset staging process before future database initialization. The goal was to explore the schema of a prepared dataset, analyze its structure, and prepare it for import into Cloud SQL through a regional storage bucket.



1. What are the names of the tables that are created?

After viewing the SQL schema using:

- cat cloudsql/table_creation.sql

the script defined three tables within the database recommendation_spark:

- Accommodation
- Rating
- Recommendation

These form the relational foundation for storing accommodation data, user ratings, and recommendation predictions.

2. What are the primary keys of each table?

Each table was created with distinct primary keys:

- Accommodation: primary key (id)
- Rating: composite primary key (accoId, userId)
- Recommendation: composite primary key (userId, accoId)

This design enforces entity uniqueness and enables foreign key relationships across the schema.

3. What data (e.g. columns) does the Accommodation table hold?

The Accommodation table holds seven columns representing accommodation metadata:

- id (varchar)
- title (varchar)
- location (varchar)
- price (int)
- rooms (int)
- rating (float)
- type (varchar)

These attributes capture the property identity, geographic context, pricing details, rating score, and accommodation type.

```
CREATE TABLE  IF NOT EXISTS Recommendation
(
  userId varchar(255),
  accoId varchar(255),
  prediction float,
  PRIMARY KEY(userId, accoId),
  FOREIGN KEY (accoId)
    REFERENCES Accommodation(id)
);

Odin ID: calizo

calizo@cloudshell:~/training-data-analyst/CPB100/lab3a (cloud-calizo-calizo)$ cat cloudsql/accommodation.csv | gre
p "Dublin"
6,Pleasant Quiet Place,Dublin,35,5,4.3,house
77,Great Private Country House,Dublin,1150,10,2.4,mansion
calizo@cloudshell:~/training-data-analyst/CPB100/lab3a (cloud-calizo-calizo)$ grep -n 'Dublin' cloudsql/accommodat
ion.csv
6:6,Pleasant Quiet Place,Dublin,35,5,4.3,house
77:77,Great Private Country House,Dublin,1150,10,2.4,mansion
calizo@cloudshell:~/training-data-analyst/CPB100/lab3a (cloud-calizo-calizo)$
```

4. Assuming the column data is ordered as in the DDL, list the attributes and their values for each accommodation in Dublin.

After searching the CSV with:

- cat cloudsql/accommodation.csv | grep "Dublin"

two accommodations were found containing the value "Dublin" in the location field:

Accommodation 1:

- id: 6
- title: Pleasant Quiet Place
- location: Dublin
- price: 35
- rooms: 5
- rating: 4.3
- type: house

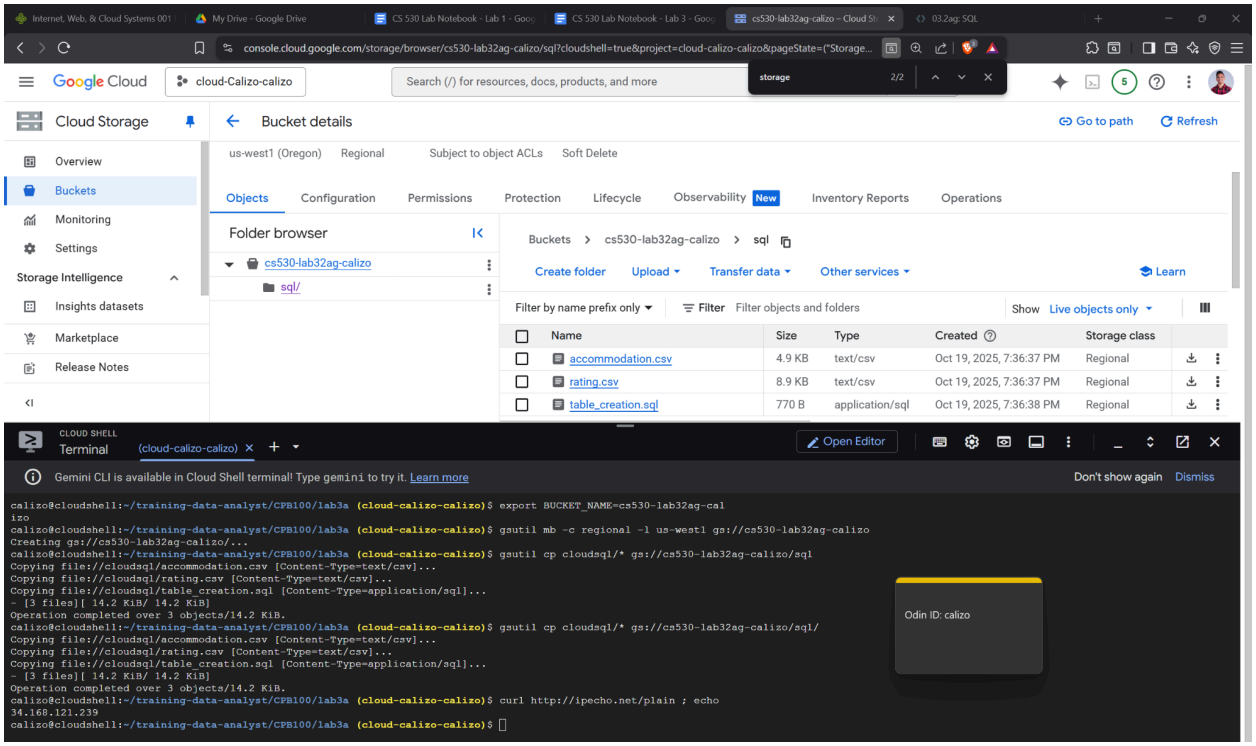
Accommodation 2:

- id: 77
- title: Great Private Country House
- location: Dublin
- price: 1150
- rooms: 10
- rating: 2.4
- type: mansion

Bucket Creation and File Staging

Next, I created a regional storage bucket in us-west1 for the SQL data transfer:

- export BUCKET_NAME=cs530-lab32ag-calizo
- gsutil mb -c regional -l us-west1 gs://cs530-lab32ag-calizo/
- gsutil cp cloudsql/* gs://cs530-lab32ag-calizo/sql/



Verification through the Cloud Console confirmed that all three files - accommodation.csv, rating.csv, and table_creation.sql - were successfully uploaded to the sql/ folder in the bucket.

Cloud Shell External IP

To prepare for future SQL access, I retrieved my Cloud Shell external IP address:

- curl http://ipecho.net/plain ; echo

This IP will be used later to configure authorized connections to the Cloud SQL instance.

Insight

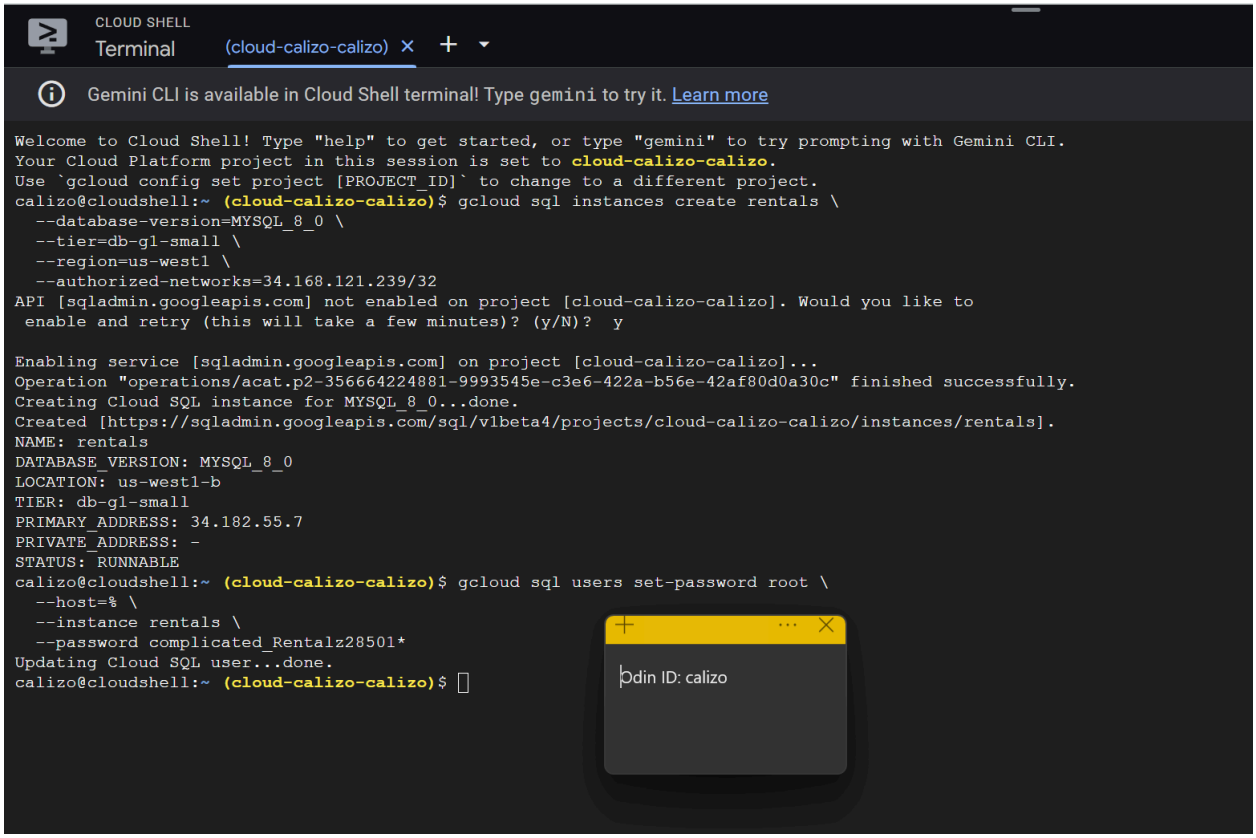
This step reinforced understanding of SQL table design and relationships while bridging it with GCP's cloud-hosted SQL environment. By analyzing schemas directly through cat and staging data into Cloud Storage, I established a working foundation for automated database deployment and querying in the upcoming lab stages.

Step 4: Cloud SQL Instance Creation (Option #1)

This step involved setting up a managed MySQL instance on Google Cloud's SQL service. Instead of using the web console, I opted for the Cloud Shell CLI method in Step 5, but the web UI process would have been nearly identical - specifying a MySQL

engine, naming the instance rentals, assigning it to us-west1-b, disabling deletion protection, and authorizing Cloud Shell's IP address for access.

Step 5: Cloud SQL Instance Creation (Option #2)



Command-Line Instance Setup

Using Cloud Shell, I created the MySQL instance with the following configuration:

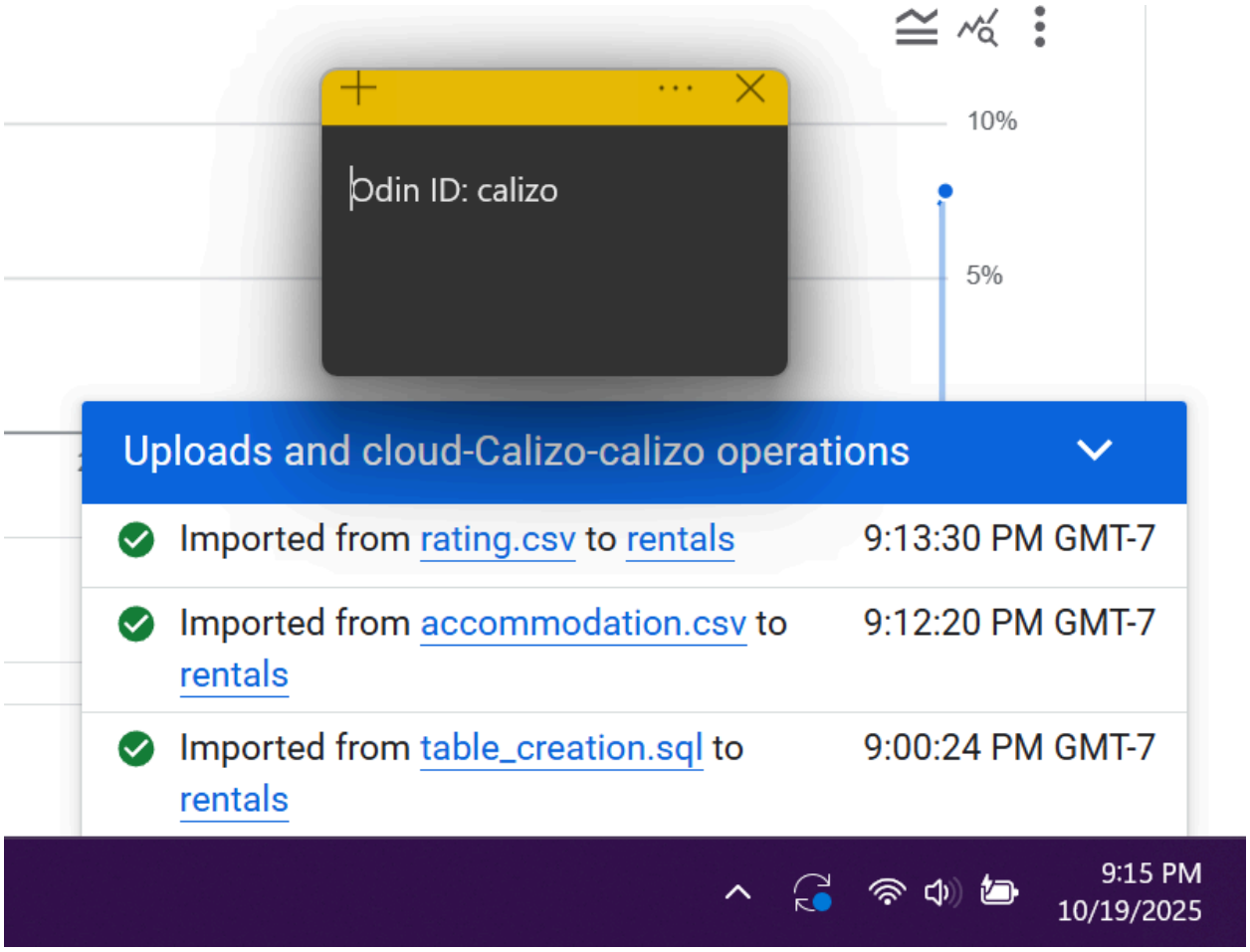
```
gcloud sql instances create rentals \
  --database-version=MYSQL_8_0 \
  --tier=db-g1-small \
  --region=us-west1 \
  --authorized-networks=34.168.121.239/32
```

When prompted, I enabled the SQL Admin API service, and the instance successfully deployed with the external IP 34.182.55.7. I then set the root password for the database using:

```
gcloud sql users set-password root \
  --host=% \
  --instance=rentals \
  --password complicated_Rentalz28501*
```

At this point, the instance rentals was active and ready for database import.

Step 6: Cloud SQL Importing Data



After confirming the SQL instance's readiness, I moved to the Cloud SQL import interface in the GCP console. From my previously created bucket, I sequentially imported three files:

1. table_creation.sql - established the database schema (recommendation_spark).
2. accommodation.csv - populated the Accommodation table with property data.
3. rating.csv - filled the Rating table with numerical user feedback data.

Each import was executed successfully, as verified by the Cloud Console activity log, confirming all files were ingested into the rentals instance. This confirmed the MySQL environment was now populated and synchronized with the structured schema from earlier steps.

Step 7: Cloud SQL from Cloud Shell

Connection and Verification
I connected to the active SQL instance using the MySQL client:

```
mysql --host=34.182.55.7 --user=root --password
```

Once logged in, I switched to the lab database:

```
use recommendation_spark;
```

and confirmed that the tables existed and data had imported properly with:

```
show tables;
select * from Rating limit 5;
```

Query Execution - Accommodation Price Ranges
I ran two price-based queries to explore data distribution:

1. **Low-end properties**

   ```
   select * from Accommodation where price < 100;
   ```

   This returned 45 entries, mostly cottages and smaller houses across locations like Vancouver, San Francisco, and London.

2. **High-end properties**

   ```
   select * from Accommodation where price > 1000;
   ```

   This yielded 36 results featuring luxurious castles and mansions, particularly concentrated in Melbourne, Paris, and Seattle.

Query Execution - Accommodation Types
Next, I explored two distinct property categories:

1. **Houses**

   ```
   select * from Accommodation where type = 'house';
   ```

   This returned 20 results, including modest family homes across major cities like NYC, Dublin, and Tokyo.

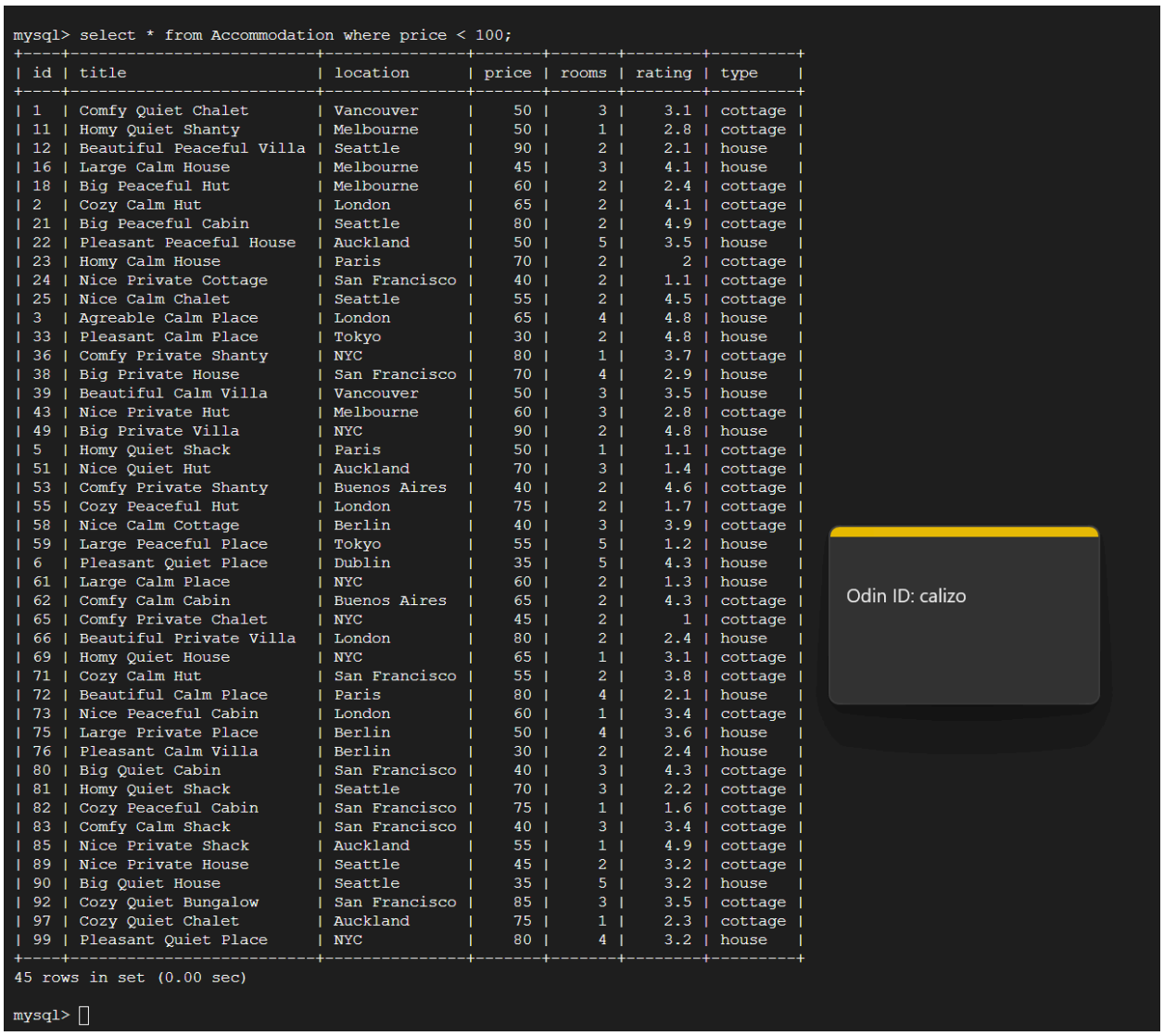
2. **Mansions**

```
select * from Accommodation where type = 'mansion';
```

This produced 18 results highlighting large estate-style residences in locales such as Auckland, San Francisco, and Melbourne.

1. Take screenshots of the output of each query for your lab notebook.

Screenshots were captured for all four queries (price < 100, price > 1000, type = 'house', and type = 'mansion') within the Cloud Shell terminal, each clearly displaying the full MySQL output and confirming accurate data retrieval from the imported dataset.



```
mysql> select * from Accommodation where price < 100;
+----+--------------------------+---------------+-------+-------+--------+---------+
| id | title                    | location      | price | rooms | rating | type    |
+----+--------------------------+---------------+-------+-------+--------+---------+
| 1  | Comfy Quiet Chalet       | Vancouver     |    50 |     3 |    3.1 | cottage |
| 11 | Homy Quiet Shanty        | Melbourne     |    50 |     1 |    2.8 | cottage |
| 12 | Beautiful Peaceful Villa | Seattle       |    90 |     2 |    2.1 | house   |
| 16 | Large Calm House         | Melbourne     |    45 |     3 |    4.1 | house   |
| 18 | Big Peaceful Hut         | Melbourne     |    60 |     2 |    2.4 | cottage |
| 2  | Cozy Calm Hut            | London        |    65 |     2 |    4.1 | cottage |
| 21 | Big Peaceful Cabin       | Seattle       |    80 |     2 |    4.9 | cottage |
| 22 | Pleasant Peaceful House  | Auckland      |    50 |     5 |    3.5 | house   |
| 23 | Homy Calm House          | Paris         |    70 |     2 |      2 | cottage |
| 24 | Nice Private Cottage     | San Francisco |    40 |     2 |    1.1 | cottage |
| 25 | Nice Calm Chalet         | Seattle       |    55 |     2 |    4.5 | cottage |
| 3  | Agreable Calm Place      | London        |    65 |     4 |    4.8 | house   |
| 33 | Pleasant Calm Place      | Tokyo         |    30 |     2 |    4.8 | house   |
| 36 | Comfy Private Shanty     | NYC           |    80 |     1 |    3.7 | cottage |
| 38 | Big Private House        | San Francisco |    70 |     4 |    2.9 | house   |
| 39 | Beautiful Calm Villa     | Vancouver     |    50 |     3 |    3.5 | house   |
| 43 | Nice Private Hut         | Melbourne     |    60 |     3 |    2.8 | cottage |
| 49 | Big Private Villa        | NYC           |    90 |     2 |    4.8 | house   |
| 5  | Homy Quiet Shack         | Paris         |    50 |     1 |    1.1 | cottage |
| 51 | Nice Quiet Hut           | Auckland      |    70 |     3 |    1.4 | cottage |
| 53 | Comfy Private Shanty     | Buenos Aires  |    40 |     2 |    4.6 | cottage |
| 55 | Cozy Peaceful Hut        | London        |    75 |     2 |    1.7 | cottage |
| 58 | Nice Calm Cottage        | Berlin        |    40 |     3 |    3.9 | cottage |
| 59 | Large Peaceful Place     | Tokyo         |    55 |     5 |    1.2 | house   |
| 6  | Pleasant Quiet Place     | Dublin        |    35 |     5 |    4.3 | house   |
| 61 | Large Calm Place         | NYC           |    60 |     2 |    1.3 | house   |
| 62 | Comfy Calm Cabin         | Buenos Aires  |    65 |     2 |    4.3 | cottage |
| 65 | Comfy Private Chalet     | NYC           |    45 |     2 |      1 | cottage |
| 66 | Beautiful Private Villa  | London        |    80 |     2 |    2.4 | house   |
| 69 | Homy Quiet House         | NYC           |    65 |     1 |    3.1 | cottage |
| 71 | Cozy Calm Hut            | San Francisco |    55 |     2 |    3.8 | cottage |
| 72 | Beautiful Calm Place     | Paris         |    80 |     4 |    2.1 | house   |
| 73 | Nice Peaceful Cabin      | London        |    60 |     1 |    3.4 | cottage |
| 75 | Large Private Place      | Berlin        |    50 |     4 |    3.6 | house   |
| 76 | Pleasant Calm Villa      | Berlin        |    30 |     2 |    2.4 | house   |
| 80 | Big Quiet Cabin          | San Francisco |    40 |     3 |    4.3 | cottage |
| 81 | Homy Quiet Shack         | Seattle       |    70 |     3 |    2.2 | cottage |
| 82 | Cozy Peaceful Cabin      | San Francisco |    75 |     1 |    1.6 | cottage |
| 83 | Comfy Calm Shack         | San Francisco |    40 |     3 |    3.4 | cottage |
| 85 | Nice Private Shack       | Auckland      |    55 |     1 |    4.9 | cottage |
| 89 | Nice Private House       | Seattle       |    45 |     2 |    3.2 | cottage |
| 90 | Big Quiet House          | Seattle       |    35 |     5 |    3.2 | house   |
| 92 | Cozy Quiet Bungalow      | San Francisco |    85 |     3 |    3.5 | cottage |
| 97 | Cozy Quiet Chalet        | Auckland      |    75 |     1 |    2.3 | cottage |
| 99 | Pleasant Quiet Place     | NYC           |    80 |     4 |    3.2 | house   |
+----+--------------------------+---------------+-------+-------+--------+---------+
45 rows in set (0.00 sec)

mysql>
```

Odin ID: calizo

```
mysql> select * from Accommodation where price > 1000;
+----+----------------------------+---------------+-------+-------+--------+---------+
| id | title                      | location      | price | rooms | rating | type    |
+----+----------------------------+---------------+-------+-------+--------+---------+
| 13 | Enormous Peaceful Fortress | Melbourne     |  3300 |    12 |    2.3 | castle  |
| 14 | Colossal Peaceful Palace   | Melbourne     |  1200 |    21 |    1.5 | castle  |
| 15 | Vast Private Fort          | London        |  1300 |    18 |    2.6 | castle  |
| 19 | Giant Quiet Castle         | Paris         |  4500 |    18 |    1.6 | castle  |
| 26 | Enormous Peaceful Palace   | Paris         |  1300 |    18 |    1.1 | castle  |
| 27 | Enormous Calm Castle       | Berlin        |  1500 |    12 |    2.3 | castle  |
| 31 | Colossal Private Castle    | Buenos Aires  |  1400 |    15 |    3.3 | castle  |
| 34 | Vast Private Fort          | NYC           |  4400 |    21 |    1.7 | castle  |
| 35 | Colossal Quiet Chateau     | NYC           |  2300 |    14 |    4.6 | castle  |
| 37 | Enormous Quiet Chateau     | Berlin        |  2000 |    20 |    2.7 | castle  |
| 4  | Colossal Quiet Chateau     | Paris         |  3400 |    16 |    2.7 | castle  |
| 40 | Colossal Private Castle    | Seattle       |  2900 |    24 |    1.5 | castle  |
| 44 | Big Peaceful Chateau       | Melbourne     |  3400 |    21 |    3.2 | castle  |
| 45 | Vast Quiet Chateau         | Tokyo         |  1100 |    19 |    2.3 | castle  |
| 46 | Colossal Private Castle    | San Francisco |  1900 |    15 |    3.7 | castle  |
| 48 | Big Calm Fort              | Vancouver     |  4500 |    22 |      4 | castle  |
| 50 | Enormous Calm Fort         | Seattle       |  2300 |    22 |    4.5 | castle  |
| 52 | Giant Private Palace       | Melbourne     |  1800 |    23 |    2.7 | castle  |
| 54 | Enormous Quiet Chateau     | Melbourne     |  4400 |    20 |    1.7 | castle  |
| 60 | Vast Peaceful Palace       | Seattle       |  1600 |    19 |    1.1 | castle  |
| 63 | Big Private Chateau        | Buenos Aires  |  2400 |    23 |    4.5 | castle  |
| 64 | Enormous Peaceful Fort     | Berlin        |  3500 |    13 |    1.8 | castle  |
| 67 | Giant Calm Chateau         | Vancouver     |  2300 |    13 |    3.2 | castle  |
| 68 | Giant Peaceful Fort        | Paris         |  1800 |    21 |    1.1 | castle  |
| 7  | Vast Peaceful Fortress     | Seattle       |  3200 |    24 |    1.9 | castle  |
| 70 | Great Calm Sately House    | Paris         |  1050 |    10 |    2.2 | mansion |
| 74 | Giant Calm Fort            | Melbourne     |  2400 |    12 |    2.3 | castle  |
| 77 | Great Private Country House| Dublin        |  1150 |    10 |    2.4 | mansion |
| 78 | Giant Private Fortress     | Tokyo         |  2100 |    17 |    2.5 | castle  |
| 79 | Large Private Manor        | Vancouver     |  1050 |    10 |    4.8 | mansion |
| 8  | Giant Quiet Fortress       | San Francisco |  3400 |    12 |    4.1 | castle  |
| 88 | Colossal Quiet Palace      | Seattle       |  4100 |    16 |    3.6 | castle  |
| 9  | Giant Peaceful Palace      | London        |  1500 |    20 |    3.5 | castle  |
| 93 | Giant Quiet Chateau        | Vancouver     |  1800 |    16 |    3.9 | castle  |
| 94 | Giant Peaceful Castle      | Auckland      |  2900 |    25 |    3.3 | castle  |
| 98 | Big Private Castle         | Paris         |  2000 |    23 |    4.6 | castle  |
+----+----------------------------+---------------+-------+-------+--------+---------+
36 rows in set (0.00 sec)

mysql>
```

Odin ID: calizo

```
mysql> select * from Accommodation where type = 'house';
+----+--------------------------+---------------+-------+-------+--------+-------+
| id | title                    | location      | price | rooms | rating | type  |
+----+--------------------------+---------------+-------+-------+--------+-------+
| 12 | Beautiful Peaceful Villa | Seattle       |    90 |     2 |    2.1 | house |
| 16 | Large Calm House         | Melbourne     |    45 |     3 |    4.1 | house |
| 22 | Pleasant Peaceful House  | Auckland      |    50 |     5 |    3.5 | house |
| 28 | Beautiful Calm Villa     | Tokyo         |   110 |     2 |    4.2 | house |
| 3  | Agreable Calm Place      | London        |    65 |     4 |    4.8 | house |
| 30 | Large Peaceful House     | Berlin        |   110 |     5 |    2.3 | house |
| 33 | Pleasant Calm Place      | Tokyo         |    30 |     2 |    4.8 | house |
| 38 | Big Private House        | San Francisco |    70 |     4 |    2.9 | house |
| 39 | Beautiful Calm Villa     | Vancouver     |    50 |     3 |    3.5 | house |
| 49 | Big Private Villa        | NYC           |    90 |     2 |    4.8 | house |
| 59 | Large Peaceful Place     | Tokyo         |    55 |     5 |    1.2 | house |
| 6  | Pleasant Quiet Place     | Dublin        |    35 |     5 |    4.3 | house |
| 61 | Large Calm Place         | NYC           |    60 |     2 |    1.3 | house |
| 66 | Beautiful Private Villa  | London        |    80 |     2 |    2.4 | house |
| 72 | Beautiful Calm Place     | Paris         |    80 |     4 |    2.1 | house |
| 75 | Large Private Place      | Berlin        |    50 |     4 |    3.6 | house |
| 76 | Pleasant Calm Villa      | Berlin        |    30 |     2 |    2.4 | house |
| 86 | Large Quiet House        | London        |   100 |     4 |      4 | house |
| 90 | Big Quiet House          | Seattle       |    35 |     5 |    3.2 | house |
| 99 | Pleasant Quiet Place     | NYC           |    80 |     4 |    3.2 | house |
+----+--------------------------+---------------+-------+-------+--------+-------+
20 rows in set (0.00 sec)

mysql>
```

Odin ID: calizo

```
mysql> select * from Accommodation where type = 'mansion';
+----+--------------------------------+---------------+-------+-------+--------+---------+
| id | title                          | location      | price | rooms | rating | type    |
+----+--------------------------------+---------------+-------+-------+--------+---------+
| 10 | Sizable Calm Country House     | Auckland      |   650 |     9 |    4.9 | mansion |
| 17 | Large Calm Sately House        | NYC           |   850 |     9 |    1.2 | mansion |
| 20 | Big Private Hall               | Buenos Aires  |   650 |    12 |    1.2 | mansion |
| 29 | Big Quiet Manor                | San Francisco |   650 |    12 |    4.3 | mansion |
| 32 | Immense Private Hall           | Seattle       |   850 |    12 |      1 | mansion |
| 41 | Big Calm Manor                 | Seattle       |   800 |    11 |    2.7 | mansion |
| 42 | Large Calm Residence           | London        |   900 |    12 |    2.4 | mansion |
| 47 | Sizable Calm Sately House      | Seattle       |   900 |    10 |    1.5 | mansion |
| 56 | Sizable Private Residence      | London        |   800 |    11 |    3.5 | mansion |
| 57 | Immense Quiet Residence        | Auckland      |   800 |    11 |    3.5 | mansion |
| 70 | Great Calm Sately House        | Paris         |  1050 |    10 |    2.2 | mansion |
| 77 | Great Private Country House    | Dublin        |  1150 |    10 |    2.4 | mansion |
| 79 | Large Private Manor            | Vancouver     |  1050 |    10 |    4.8 | mansion |
| 84 | Great Peaceful Sately House    | Melbourne     |   700 |     8 |    3.2 | mansion |
| 87 | Immense Peaceful Hall          | San Francisco |   850 |    12 |    4.4 | mansion |
| 91 | Large Peaceful Hall            | Melbourne     |   650 |    10 |    1.9 | mansion |
| 95 | Great Calm Hall                | San Francisco |   800 |    11 |    3.8 | mansion |
| 96 | Immense Private Country House  | Tokyo         |   800 |     9 |    3.8 | mansion |
+----+--------------------------------+---------------+-------+-------+--------+---------+
18 rows in set (0.00 sec)

mysql>
```

Odin ID: calizo

Insight

Through Steps 4-7, I successfully deployed a MySQL instance named rentals on GCP, imported preconfigured datasets into a managed database, and confirmed data integrity using direct SQL queries. The sequence demonstrated the seamless integration between Cloud Storage and Cloud SQL, transitioning from schema definition to live querying. The exercise reinforced both the conceptual and practical understanding of connecting cloud-based storage, secure authentication, and SQL-level data validation.

Step 8: Cloud SQL Cleanup

To conclude the SQL lab environment setup, I performed a complete teardown of the storage bucket and Cloud SQL instance created earlier. This ensures that no unnecessary resources remain active, maintaining a clean and cost-free cloud workspace.

I began by deleting the regional Cloud Storage bucket that housed the schema and data files from earlier steps. The command used was:

- gsutil rm -r gs://cs530-lab32ag-calizo

The terminal confirmed the deletion of all three objects (accommodation.csv, rating.csv, and table_creation.sql), verifying that the bucket and its contents had been fully removed.

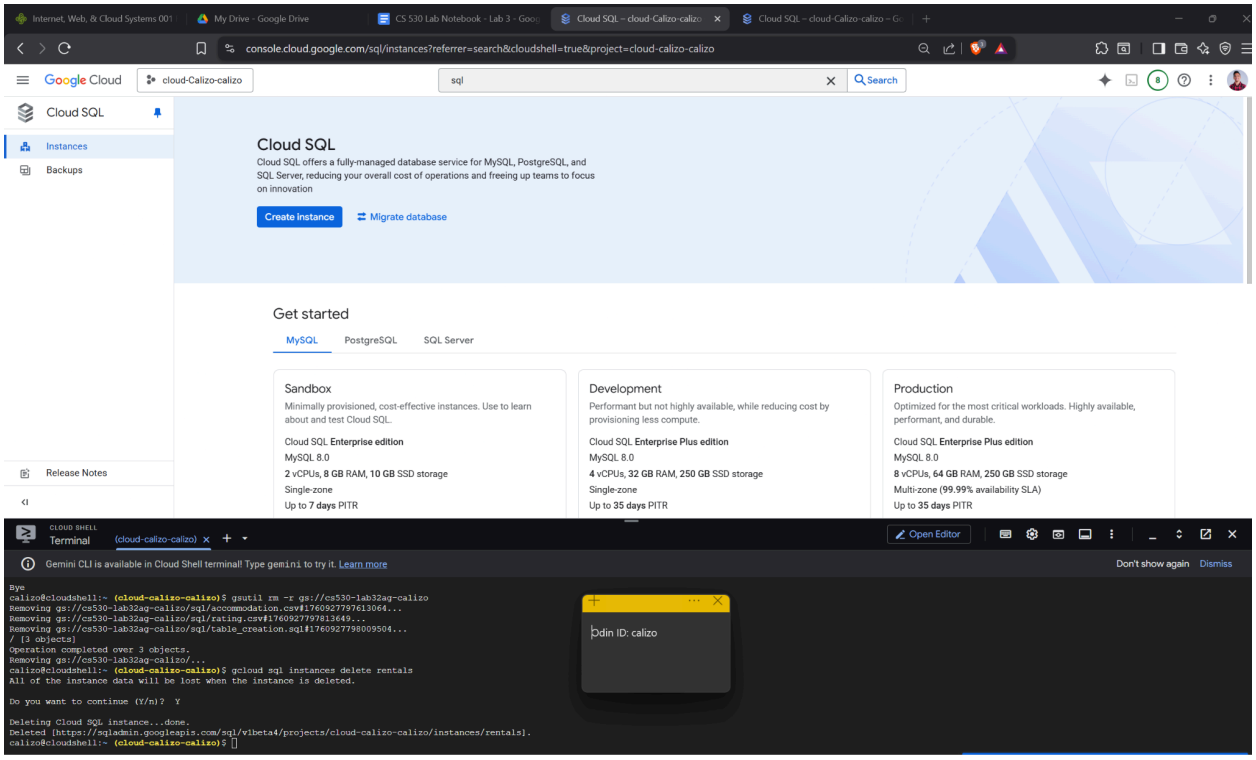
Next, I deleted the MySQL instance rentals using the following command:

- gcloud sql instances delete rentals

After confirming the prompt, the deletion process completed successfully, with Cloud Shell reporting the instance removal under the SQL Admin API logs.

To verify the cleanup, I navigated to the Cloud SQL Instances page in the Google Cloud Console. The screen displayed the default Cloud SQL landing page with no active instances, confirming that the rentals instance was no longer present and the environment had been fully reset.

1. Take a screenshot showing no instances are running.



A screenshot was captured displaying the Cloud SQL interface with zero active instances alongside the terminal logs of successful bucket and instance deletion, completing the cleanup process for Lab 3.2ag.

<u>Insight</u>

This final cleanup step highlighted the importance of lifecycle management in cloud resource provisioning. Beyond building and querying databases, responsibly decommissioning unused services demonstrates sound DevOps and cost-control

practices - ensuring the cloud remains a efficient environment for future deployments.

Step 9: AWS RDS

To extend the lab's database learning into AWS, I began by launching the AWS Management Console and navigating to the RDS service. This environment mirrors the relational database setup from Cloud SQL but uses Amazon's managed infrastructure for MySQL. The goal here was to experience firsthand how cloud providers simplify SQL deployment through managed services.

Step 10: Cloud Shell Network Address

Using AWS Cloud Shell, I retrieved my session's public IP address. This would serve as the entry point for later MySQL connections.

- curl http://ipecho.net/plain ; echo

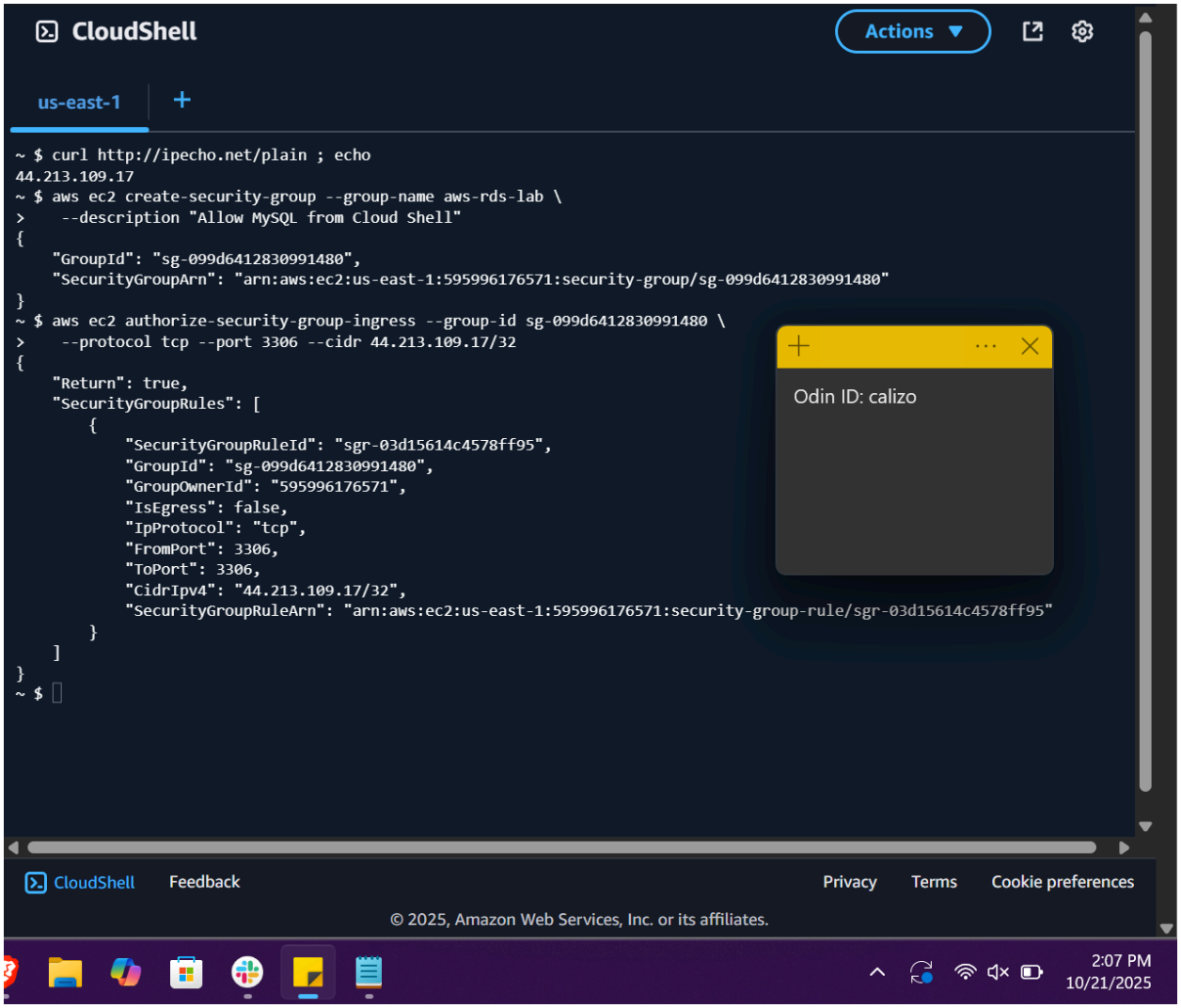
The returned IP address was then used to configure a CIDR rule for database access.

Step 11: RDS Security Group Creation

In Cloud Shell, I executed two commands to create and configure the security group for MySQL traffic:

```
aws ec2 create-security-group --group-name aws-rds-lab \
  --description "Allow MySQL from Cloud Shell"

aws ec2 authorize-security-group-ingress --group-id <GroupId> \
  --protocol tcp --port 3306 --cidr <CloudShell_IP>/32
```

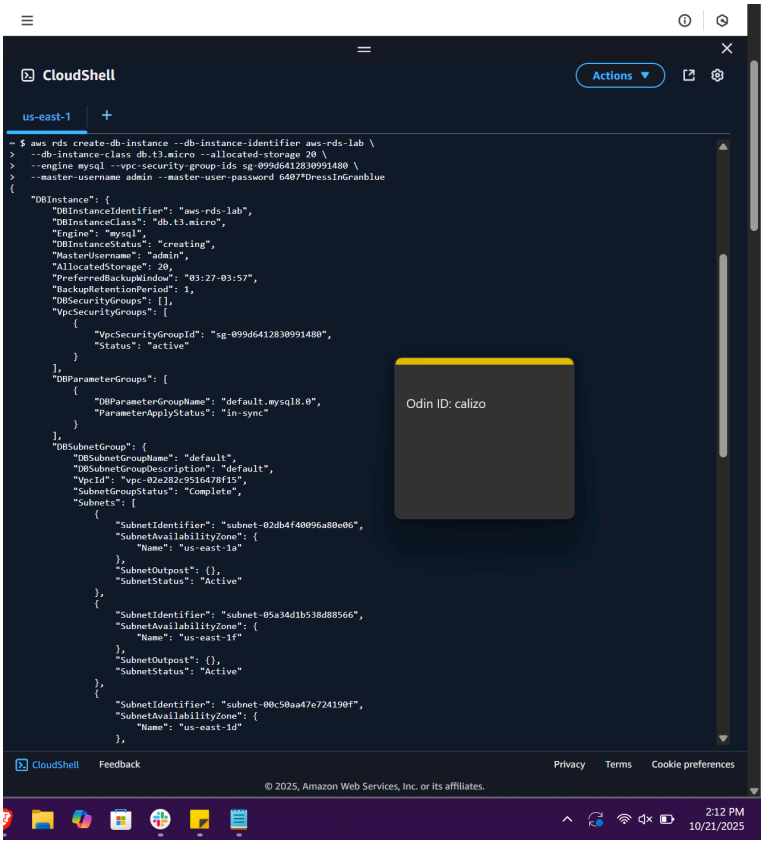


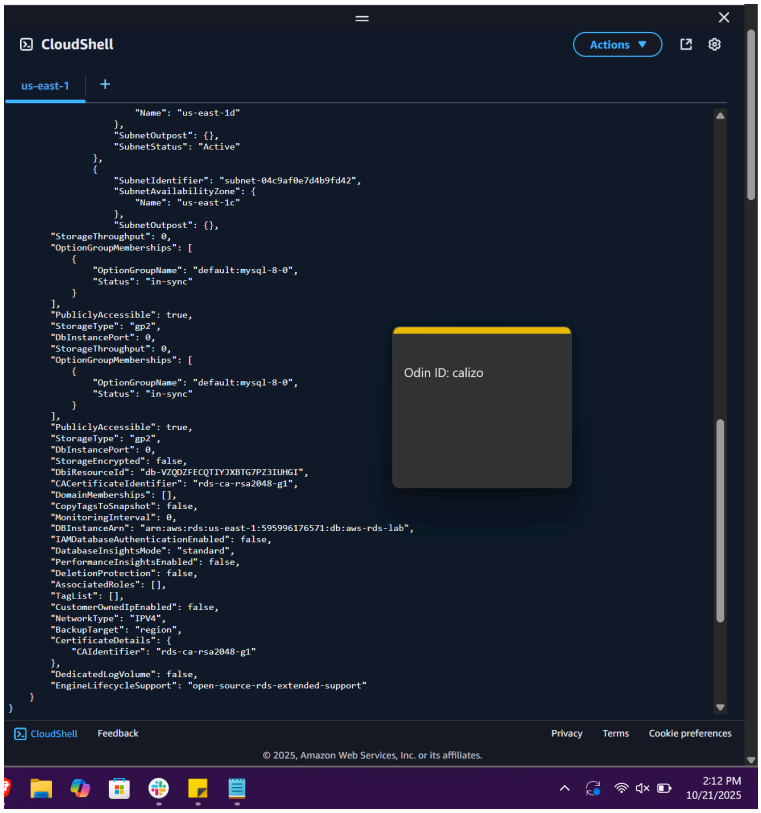
The output confirmed that the new group (with ID sg-099d6412830991480) was successfully created and granted inbound MySQL access for port 3306 from my Cloud Shell IP address. This step ensured controlled and secure connectivity between AWS RDS and my session environment.

Step 12: RDS Instance Creation

With networking configured, I created a new RDS instance named aws-rds-lab directly from the AWS CLI:

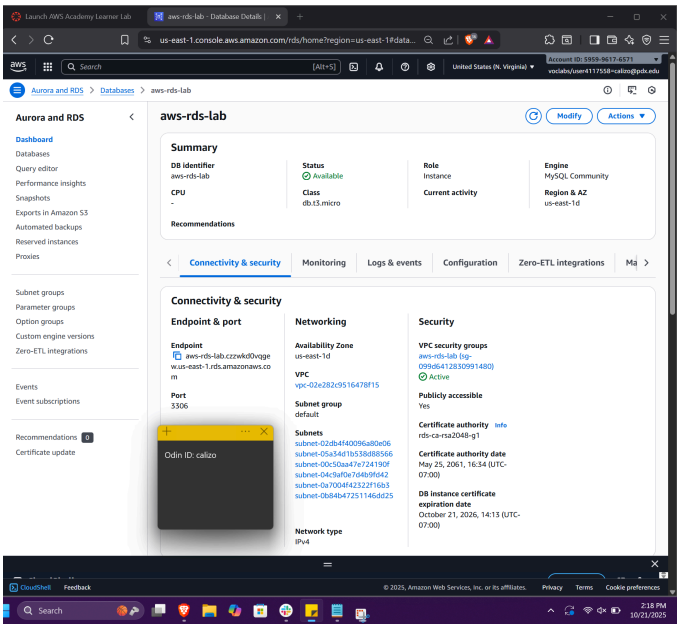
```
aws rds create-db-instance --db-instance-identifier aws-rds-lab \
  --db-instance-class db.t3.micro --allocated-storage 20 \
  --engine mysql --vpc-security-group-ids sg-099d6412830991480 \
  --master-username admin --master-user-password 6407*DressInGranblue
```


CloudShell
Actions
us-east-1
~ $ aws rds create-db-instance --db-instance-identifier aws-rds-lab \
> --db-instance-class db.t3.micro --allocated-storage 20 \
> --engine mysql --vpc-security-group-ids sg-099d641283099l480 \
> --master-username admin --master-user-password 6407*DressInGranblue
{
"DBInstance": {
"DBInstanceIdentifier": "aws-rds-lab",
"DBInstanceClass": "db.t3.micro",
"Engine": "mysql",
"DBInstanceStatus": "creating",
"MasterUsername": "admin",
"AllocatedStorage": 20,
"PreferredBackupWindow": "03:27-03:57",
"BackupRetentionPeriod": 1,
"DBSecurityGroups": [],
"VpcSecurityGroups": [
{
"VpcSecurityGroupId": "sg-099d641283099l480",
"Status": "active"
}
],
"DBParameterGroups": [
{
"DBParameterGroupName": "default.mysql8.0",
"ParameterApplyStatus": "in-sync"
}
],
"DBSubnetGroup": {
"DBSubnetGroupName": "default",
"DBSubnetGroupDescription": "default",
"VpcId": "vpc-02e282c9516478f15",
"SubnetGroupStatus": "Complete",
"Subnets": [
{
"SubnetIdentifier": "subnet-02db4f40096a80e06",
"SubnetAvailabilityZone": {
"Name": "us-east-1a"
},
"SubnetOutpost": {},
"SubnetStatus": "Active"
},
{
"SubnetIdentifier": "subnet-05a34d1b538d88566",
"SubnetAvailabilityZone": {
"Name": "us-east-1f"
},
"SubnetOutpost": {},
"SubnetStatus": "Active"
},
{
"SubnetIdentifier": "subnet-00c50aa47e724190f",
"SubnetAvailabilityZone": {
"Name": "us-east-1d"
},
Odin ID: calizo
CloudShell
Feedback
Privacy
Terms
Cookie preferences
© 2025, Amazon Web Services, Inc. or its affiliates.
2:12 PM
10/21/2025


CloudShell
Actions
us-east-1
"Name": "us-east-1d"
},
"SubnetOutpost": {},
"SubnetStatus": "Active"
},
{
"SubnetIdentifier": "subnet-04c9af0e7d4b9fd42",
"SubnetAvailabilityZone": {
"Name": "us-east-1c"
},
"SubnetOutpost": {},
"StorageThroughput": 0,
"OptionGroupMemberships": [
{
"OptionGroupName": "default:mysql-8-0",
"Status": "in-sync"
}
],
"PubliclyAccessible": true,
"StorageType": "gp2",
"DbInstancePort": 0,
"StorageThroughput": 0,
"OptionGroupMemberships": [
{
"OptionGroupName": "default:mysql-8-0",
"Status": "in-sync"
}
],
"PubliclyAccessible": true,
"StorageType": "gp2",
"DbInstancePort": 0,
"StorageEncrypted": false,
"DbiResourceId": "db-VZQDZFECQTIYJXBTG7PZ3IUHGI",
"CACertificateIdentifier": "rds-ca-rsa2048-g1",
"DomainMemberships": [],
"CopyTagsToSnapshot": false,
"MonitoringInterval": 0,
"DBInstanceArn": "arn:aws:rds:us-east-1:595996176571:db:aws-rds-lab",
"IAMDatabaseAuthenticationEnabled": false,
"DatabaseInsightsMode": "standard",
"PerformanceInsightsEnabled": false,
"DeletionProtection": false,
"AssociatedRoles": [],
"TagList": [],
"CustomerOwnedIpEnabled": false,
"NetworkType": "IPV4",
"BackupTarget": "region",
"CertificateDetails": {
"CAIdentifier": "rds-ca-rsa2048-g1"
},
"DedicatedLogVolume": false,
"EngineLifecycleSupport": "open-source-rds-extended-support"
}
}
Odin ID: calizo
CloudShell
Feedback
Privacy
Terms
Cookie preferences
© 2025, Amazon Web Services, Inc. or its affiliates.
2:12 PM
10/21/2025

The command output confirmed the instance was Publicly Accessible with the MySQL 8.0 engine and placed in the us-east-1d availability zone. The creation process encapsulated all necessary configurations, from subnets and storage to accessibility settings.

Step 13: Viewing the Instance



Once the instance reached Available status, I verified it through the RDS dashboard. The Connectivity & Security section displayed the endpoint and port details:

- Endpoint: aws-rds-lab.czzwkd0vqgew.us-east-1.rds.amazonaws.com
- Port: 3306
- VPC Security Group: aws-rds-lab (sg-099d6412830991480)

This endpoint acted as the DNS-based address for database access, enabling a seamless bridge between AWS RDS and Cloud Shell.

Step 14: RDS Test Connection

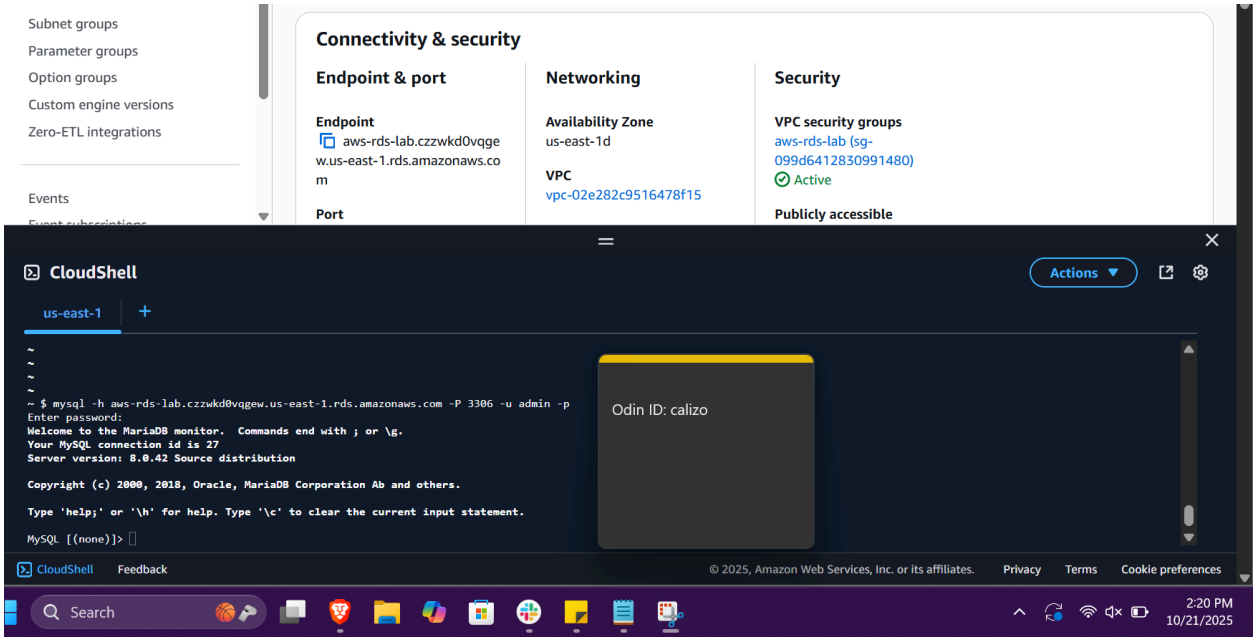
With the endpoint confirmed, I tested the MySQL connection directly from Cloud Shell using the following command:

- mysql -h aws-rds-lab.czzwkd0vqgew.us-east-1.rds.amazonaws.com -P 3306 -u admin -p

After entering the password, the MySQL monitor launched successfully, displaying the version information for MySQL 8.0.42. The connection validated both the accessibility of the database and the correctness of the security group configuration.

Step 15: RDS Cleanup

1. Show a screenshot of the successful connection similar to below that includes your OdinID.



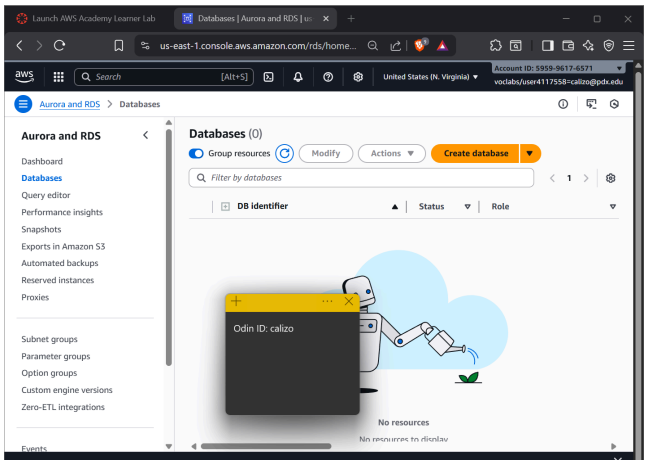
The screenshot captured my Cloud Shell terminal confirming the connection to the AWS RDS instance, with the MySQL prompt (mysql>) visible, indicating a successful handshake.

Step 16: RDS Cleanup

Following successful connection and verification, I removed the database resources to avoid unnecessary costs and to maintain a clean AWS environment.

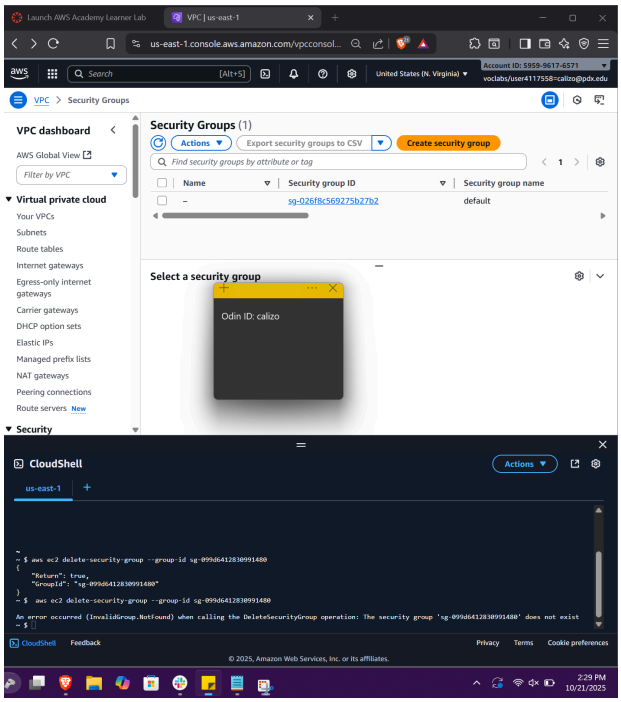
First, I deleted the RDS instance:

```
aws rds delete-db-instance --db-instance-identifier aws-rds-lab \
  --skip-final-snapshot
```



After AWS confirmed the deletion, I removed the security group:

aws ec2 delete-security-group --group-id sg-099d6412830991480



The console returned confirmation that both operations completed successfully. Attempting a second deletion verified the group no longer existed, confirming proper cleanup.

Lastly, I revisited both the RDS Dashboard and VPC Security Groups sections of the AWS Console. The RDS Databases page showed zero running instances, while the Security Groups view displayed only the default VPC configuration - proof that all custom configurations were removed.

Insight
Steps 9 through 16 illustrated how AWS RDS parallels Google Cloud SQL in both design and management philosophy. Through the process of provisioning, connecting, and cleaning up a MySQL instance, the lab emphasized secure cloud database lifecycle management. By using the AWS CLI and Cloud Shell together, I deepened my understanding of automation, IP-based security, and the operational symmetry of cloud database services.

Lab 3.3 sqlite3 Guestbook

Step 1: sqlite3 Guestbook Overview

This lab transitioned the Guestbook web app from an in-memory Python list to a persistent sqlite3 backend, ensuring that entries survive across server restarts. The directory 02_mvp_modules_sqlite3 was explored, showing a clear Model-View-Presenter (MVP) architecture with separate modules for logic (gbmodel), interface (index.py, sign.py), and presentation (templates/). This modular design makes the application maintainable and scalable, while the sqlite3 integration gives it the foundational persistence layer common to production systems.

Step 2: gbmodel Package

The gbmodel package acts as the modular bridge between Flask and the data layer. Inside __init__.py, the model backend was configured as "sqlite3", invoking model_sqlite3.py. Its constructor verified whether the guestbook table existed within entries.db. If absent, it created the table with columns for name, email, signed_on, and message. This small initialization block encapsulates core database setup logic in a clean, self-contained package.

Step 3: Presenter Architecture

Following the MVP pattern, the application separated the view logic into presenters. The main app (app.py) mapped URL routes to presenters using Flask's MethodView.

- The Index presenter (index.py) handled GET requests to render all entries.
- The Sign presenter (sign.py) handled both GET (form display) and POST (form submission).
    - When a user signed the guestbook, the form data was processed by gbmodel.get_model(), inserting a new record into the sqlite3 database before redirecting back to the homepage. This division between

controller and model not only clarified responsibilities but also improved testability and modularity.

Step 4: Running the Application

After activating a Python virtual environment and installing dependencies from requirements.txt, the Flask application was launched via:

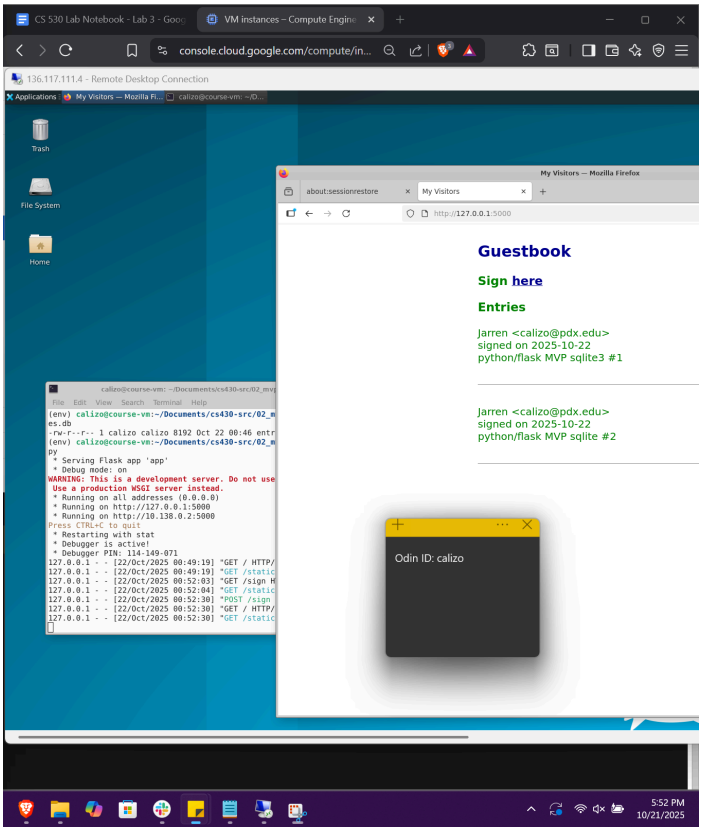
```
python app.py
```

Upon visiting http://127.0.0.1:5000/, two distinct guestbook entries were submitted:

- python/flask MVP sqlite3 #1
- python/flask MVP sqlite3 #2

These entries, each containing the PSU email and message text, persisted across server restarts, confirming that entries.db successfully maintained its data.

1. Take a screenshot of the resulting page for your lab notebook.



The screenshot shows both entries displayed under "Entries" in the Guestbook page, demonstrating data persistence after restarting the Flask server and verifying sqlite3 integration worked correctly.

Step 5: sqlite3 Database Verification

To manually inspect the backend, the sqlite3 CLI tool was used:

```
sqlite3 entries.db
```

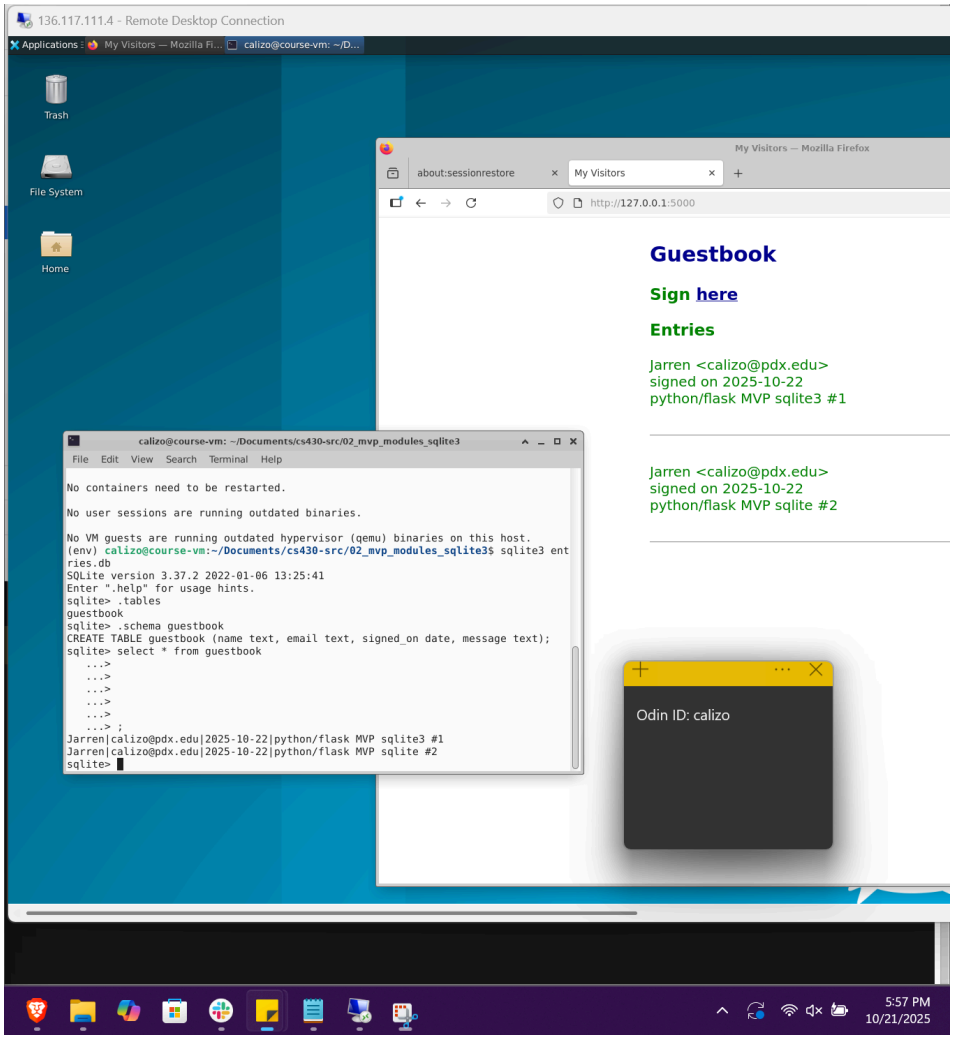
Inside the client, the following commands were executed:

```
.tables
.schema guestbook
select * from guestbook;
```

The output confirmed:

- The existence of the guestbook table.
- Its schema:
    - CREATE TABLE guestbook (name text, email text, signed_on date, message text);
- Two persisted rows matching the web entries:
    - Jarren|calizo@pdx.edu|2025-10-22|python/flask MVP sqlite3 #1
    - Jarren|calizo@pdx.edu|2025-10-22|python/flask MVP sqlite3 #2

1. Then, within the sqlite client, perform the following commands and take a screenshot of their output to include in your lab notebook.



The screenshot of the sqlite3 terminal output confirms the schema, table name, and both persisted records in the database.

<u>Insight</u>

This lab bridged the concepts of persistence and modularization. By migrating from in-memory data storage to a sqlite3-backed Flask MVP app, the project illustrated how even a lightweight database introduces reliability and real-world scalability. The dual verification - browser output and database inspection - validated a seamless flow between the frontend, presenter logic, and data persistence layer.